

# CineVerse

Redis Lock + PostgreSQL Concurrency Control  
Interview & Implementation Notes

## 1. Core Mental Model

Redis locking and PostgreSQL concurrency control are two different layers. Redis coordinates which request gets to try first; PostgreSQL remains responsible for the final correctness decision.

```
10,000 requests
↓
[ Redis Lock – Distributed Coordination ]
↓
[ PostgreSQL – Final Correctness ]
```

## 2. What Redis Lock Does

For a particular seat, Redis allows only one request to acquire the distributed lock at a time. The other competing requests can fail fast without reaching PostgreSQL.

```
10,000 requests
↓
Redis lock: lock:show:1:seat:1
↓
9,999 → lock failure → reject
1 → lock success → PostgreSQL
```

**Important:** Redis does not permanently mean that only one request can ever reach the database. It means one request can hold the lock for that seat at a time.

## 3. What If the Redis Lock Owner Fails?

Redis locks should use a TTL. For example, with a 5-second TTL, if the application crashes after acquiring the lock and cannot release it, the lock eventually expires and another request can try.

```
Request A → Redis LOCK ■ → PostgreSQL → failure/crash
↓
TTL expires
↓
Lock disappears
↓
Request B → Redis LOCK ■ → PostgreSQL
```

When the application can detect completion or failure normally, it should release the lock immediately. The TTL is the safety mechanism for cases such as crashes or lost execution.

## 4. What If PostgreSQL Rejects the Request?

Redis does not decide whether the seat is actually available. PostgreSQL remains the source of truth.

```
Redis lock acquired
↓
PostgreSQL
↓
Is seat AVAILABLE?
■ ■
YES NO
↓ ↓
AVAILABLE → HELD Reject
↓
```

## Release Redis lock

If PostgreSQL returns **updatedRows = 0**, the request failed the database state condition. The application should return failure and release the Redis lock.

## 5. Redis Lock vs Database Concurrency Control

| Mechanism                 | Role                                            | Main Question                                         |
|---------------------------|-------------------------------------------------|-------------------------------------------------------|
| Redis distributed lock    | Distributed coordination / contention reduction | Who gets to try first?                                |
| Atomic conditional UPDATE | Database state transition / correctness         | Can this seat actually change from AVAILABLE to HELD? |
| Optimistic locking        | Version-based concurrency control               | Has another transaction changed this row/version?     |
| Pessimistic locking       | Explicit database row locking                   | Can I lock this row while I perform the operation?    |

## 6. CineVerse Current DB Mechanism

The current Phase 6 DB-only implementation primarily uses an atomic conditional UPDATE:

```
UPDATE show_seats
SET status = 'HELD'
WHERE show_seat_id = ?
AND status = 'AVAILABLE';
```

This should not automatically be called optimistic locking or pessimistic locking. It is an atomic conditional state transition. Optimistic and pessimistic locking are separate database concurrency-control mechanisms.

## 7. Final CineVerse Architecture Being Tested

[illegible]

## 8. Interview Answer

Instead of saying: **“I use two locks.”**

Say:

“I use Redis as a distributed coordination and locking layer to reduce contention, while PostgreSQL uses an atomic conditional state transition as the final correctness mechanism. Redis controls which request gets to try; PostgreSQL decides whether the seat actually changes state.”

## 9. One-Line Mental Model

**Redis controls who gets to try. PostgreSQL controls whether the booking actually succeeds.**

Redis is therefore an optimization/coordination layer, not a replacement for PostgreSQL as the authoritative seat of state.